GRADIENT PROJECTION METHODS FOR CONTINUAL LEARNING IN HYPERNETWORKS

OVERCOMING CHUNK-INDUCED GRADIENT PATHOLOGIES VIA PROJECTION NORMALIZATION, AND THE INTRODUCTION OF EFOPNG

JAKUB MICHAŁOWSKI

THESIS SUBMITTED IN PARTIAL FULFILLMENT
OF THE REQUIREMENTS FOR THE DEGREE OF
BACHELOR OF SCIENCE IN COGNITIVE SCIENCE & ARTIFICIAL INTELLIGENCE

DEPARTMENT OF
COGNITIVE SCIENCE & ARTIFICIAL INTELLIGENCE
SCHOOL OF HUMANITIES AND DIGITAL SCIENCES
TILBURG UNIVERSITY

STUDENT NUMBER

u253954

COMMITTEE

dr. Giacomo Spiegler
dr. The Second Reader

LOCATION

Tilburg University
School of Humanities and Digital Sciences
Research Center of Cognitive Science &
Artificial Intelligence
Tilburg, The Netherlands

DATE

May 28, 2026

WORD COUNT

≈7800

ACKNOWLEDGMENTS

The author thanks dr. Giacomo Spiegler for supervision and feedback throughout this project.

GRADIENT PROJECTION METHODS FOR CONTINUAL LEARNING IN HYPERNETWORKS

OVERCOMING CHUNK-INDUCED GRADIENT PATHOLOGIES VIA PROJECTION NORMALIZATION, AND THE INTRODUCTION OF EFOPNG

JAKUB MICHAŁOWSKI

Abstract

This thesis resolves the fundamental incompatibility of applying gradient projection-based continual learning (CL) methods to hypernetwork architectures, establishing a novel framework for stable subspace optimization. Traditional projection methods collapse due to a structural–numerical pathology where the architectural necessity of weight-chunking—required to avoid an output parameter explosion—forces the model to be produced in hundreds or thousands of sequential chunks before a single projection step can execute. This repetitive chunk-generation loop accumulates an exceptionally large gradient volume within the memory matrix, blowing up the condition number of the projection correlation matrix to infinity and causing catastrophic inversion failures that the standard regularization parameter λ cannot rescue.

The primary research objective is to eliminate these numerical singularities and systematically evaluate whether combining hard geometric projections on top of task-conditioned hypernetworks and their output-scoped functional regularizers yields a constructive synergy. To achieve this, we introduce *projection-scoped normalization*: gradient columns stored in the memory matrix G are rescaled to unit ℓ_2 norm, and the diagonal empirical Fisher vector is rescaled by its absolute maximum, together stabilizing the metric tensor within the local projection algebra. We also introduce `eFOPNG`, an elastic variant of Fisher-Orthogonal Projected Natural Gradient Descent (Garg, Kolhe, Peng, & Gopalam, n.d.) that replaces the current-task Fisher F_{new} with the combined curvature $F_c = F_{\text{new}} + F_{\text{old}}$, inspired by the parameter-inertia concept of Elastic Weight Consolidation. Methods are benchmarked on sequential Split-MNIST and Split-CIFAR10 streams. The empirical findings reveal that projection normalization completely resolves matrix inversion collapse, and that in the hypernetwork setting all normalized projection methods outperform plain

SGD, with `eFOPNG` matching the best projection baseline. However, a standalone hypernetwork optimized with `Adam` and von Oswald's functional regularizer consistently surpasses all projection variants, indicating that momentum—which projection methods forgo by design—is the decisive factor in compressed generative weight spaces.

1 DATA SOURCE, ETHICS, CODE, AND TECHNOLOGY STATEMENT

Data Source. The benchmarking datasets used in this thesis, Split-MNIST and Split-CIFAR10, are openly accessible public repositories curated for machine learning research. The MNIST dataset is originally owned by Yann LeCun, Corinna Cortes, and Christopher J.C. Burges; the CIFAR-10 dataset is owned by Alex Krizhevsky, Vinod Nair, and Geoffrey Hinton. Both datasets are completely anonymized and licensed for public research use. This thesis did not involve collecting data from human participants or animals. Data was acquired programmatically through the open-source `torchvision.datasets` Python API.

FIGURES. All data visualizations and experimental trajectory plots were created entirely by the author using `matplotlib`, with run logs compiled via the `Weights & Biases` (W&B) platform.

CODE. The foundational principles and mathematical framework of the baseline FOPNG method were adapted from the methodology described by Garg et al. (n.d.). The complete execution infrastructure, the projection-scoped normalization layers, and the novel `eFOPNG` optimizer were implemented entirely by the author. The complete repository is hosted on GitHub at `https://github.com/[YOUR_USERNAME]/[YOUR_REPO]`. The software pipeline uses Python (v3.10) and relies on: PyTorch (v2.x), Torchvision, Weights & Biases, NumPy, Matplotlib, and Tqdm.

TECHNOLOGY. Native LaTeX compiler utilities were used exclusively to typeset the manuscript, with reference management handled natively via BibTeX. During the preparation of this work the author used Gemini (Gemini 3 Flash, Google) in order to optimize academic phrasing, assist with structural outlines, paraphrase complex technical transitions, and perform spelling and grammar checks. After using this tool, the author reviewed and edited the content as needed and takes full responsibility for the content of the thesis. No other external paraphrasing tools or academic language banks were used.

2 INTRODUCTION

The capacity for continuous, sequential adaptation without the eradication of historical knowledge remains an architectural cornerstone of biological intelligence, yet it presents a fundamental barrier for artificial neural networks. In deep learning paradigms, sequential training across non-stationary data distributions triggers a systemic pathology known as *catastrophic forgetting* (**?**). When a network optimizes its parameters for an incoming task, gradient updates blindly overwrite the weight configurations essential for retaining prior task spaces, shifting the internal representation manifolds away from historical objective minima. Modern continual learning (CL) literature is conventionally structured into three core algorithmic families: replay-based strategies that leverage episodic data buffers to rehearse historical distributions, regularization-based frameworks that penalize parameter deviations along important directions, and architectural or parameter-isolation methods (De Lange et al., n.d.). Within the meta-learning domain, Hypernetworks (**?**) have emerged as an elegant paradigm. Hypernetworks circumvent parameter-space conflicts by utilizing a centralized, lightweight meta-model to generate the target network's complete parameter weights conditioned on task embeddings, as formalized by Oswald, Henning, Grewe, and Sacramento (n.d.). Concurrently, within the field of geometric optimization, gradient projection methods—exemplified by the recently proposed Fisher-Orthogonal Projected Natural Gradient Descent (FOPNG) (Garg et al., n.d.)—explicitly constrain subsequent updates to the null-space of historical parameter spaces.

Despite the mathematical elegance of both hypernetworks and gradient projection methods, their structural intersection remains almost entirely unexamined. Crucially, this investigation uncovers a critical hidden optimization pathology born at this intersection. To prevent a catastrophic parameter explosion within the meta-model, a hypernetwork must use sequential weight-chunking; generating the entire target weight array simultaneously would require an absurdly large output layer, defeating the purpose of a compact meta-learner. Depending on the dimensional footprint of the target architecture, this constraint forces the model to be produced in hundreds or thousands of individual chunks. Running this sub-block spawning mechanism completely prior to a single execution step accumulates gradients additively, resulting in parameter gradients of exceptionally high magnitude. This explosion completely erases the stabilizing effect of λ during matrix inversion, blowing up the condition number and inducing catastrophic matrix inversion failures.

The primary research objective of this thesis is to systematically resolve this linear-algebraic collapse, map the initialization mechanics of meta-projection pipelines, and evaluate whether the integration of hard geometric constraints on top of soft functional regularizers yields an optimized sequential learning system. To achieve this, we introduce an algorithmic framework that unifies three core elements: (i) a local projection-scoped normalization mechanism that scales gradient vectors and empirical Fisher Information Matrix (FIM) components strictly within the projection scope; (ii) a pure decoupled initialization protocol via `AdamW` to prevent baseline gradient contamination; and (iii) Elastic FOPNG (`eFOPNG`), a custom projection variant inspired by the concept of parameter inertia from EWC. Our framework is benchmarked across sequential multi-head streams using Split-MNIST and Split-CIFAR10 distributions.

The scientific relevance of this thesis lies in identifying and resolving numerical optimization anomalies at the intersection of gradient projection methods and chunked hypernetworks. First, we demonstrate that additive gradient accumulation across thousands of sequential sub-chunks creates extreme gradient magnitudes that numerically erase the stabilizing effect of λ during matrix inversion (Garg et al., n.d.). We resolve this by developing projection-scoped normalization: column-wise ℓ_2 normalization of the gradient matrix G and absolute-maximum scaling of the diagonal empirical Fisher vector. Second, this work establishes a rigorous baseline for initializing generative geometric subspaces, demonstrating that `AdamW` eliminates the structural gradient contamination induced by standard `Adam` with L_2 regularization. Third, we introduce `eFOPNG` as a novel algorithmic variant that replaces the rigid orthogonal boundary of FOPNG with an elastic curvature boundary formed by the combined Fisher $F_c = F_{\text{new}} + F_{\text{old}}$.

The societal relevance of optimizing these architectures extends to the real-world deployment of edge computing, autonomous robotics, and localized medical diagnostics. Enabling compact, VRAM-efficient models to permanently acquire sequential streams of knowledge on-device—without access to centralized historical data—provides a viable pathway toward secure, sustainable artificial intelligence.

To systematically navigate these boundaries, this thesis addresses the following primary research question:

> *To what extent can gradient projection methods be successfully integrated into chunked hypernetwork architectures for continual learning, and how do their structural pathologies, initialization protocols, and elastic variants impact optimization efficiency?*

This is decomposed into four sub-research questions (Sub-RQs):

Sub-RQ1: *Does the joint integration of gradient projection frameworks and soft output-scoped functional regularization yield a constructive synergy, or does a standalone regularized hypernetwork offer a more efficient optimization frontier?*

Sub-RQ2: *How does the architectural necessity of spawning thousands of sequential weight-chunks impact gradient magnitudes, and to what extent can local projection-scoped normalization prevent the resulting explosion from collapsing the matrix inversion?*

Sub-RQ3: *To what degree does `eFOPNG`—which replaces the rigid FOPNG projection boundary with a combined Fisher preconditioner $F_c = F_{new} + F_{old}$—influence task retention compared to rigid projection baselines?*

Sub-RQ4 *(Fisher Accumulation Operator).* Given an elastic Fisher preconditioner, how should the historical component F_{old} be accumulated across tasks? Specifically, does the non-decaying maximum operator $F_{\text{old}} \leftarrow \max(F_{\text{old}}, F_{\text{new}})$ preserve early-task knowledge more effectively than a decaying exponential moving average, and does its monotonic growth impose a measurable plasticity cost over long task sequences?

The empirical findings reveal a definitive structural boundary: projection normalization completely resolves matrix inversion collapse; `AdamW` initialization establishes a cleaner base manifold than `Adam`; all projection variants surpass plain SGD in the hypernetwork setting, confirming a partial constructive synergy. However, a standalone hypernetwork optimized via `Adam` with von Oswald's functional regularizer consistently matches or outperforms all projection variants. This indicates that the momentum mechanism—which projection optimizers forgo by design because the projection step discards the optimizer state—is the decisive factor in compressed generative weight spaces.

3 RELATED WORK

This section builds the theoretical and empirical context for the thesis, tracing a path from foundational continual learning challenges through regularization and projection methods, to hypernetwork architectures and the structural gap at their intersection.

3.1 *Continual Learning: Taxonomy and the Stability-Plasticity Dilemma*

Modern machine learning systems are predominantly trained on static, i.i.d. datasets — a condition that rarely holds in deployment, where agents must adapt to non-stationary data streams while retaining prior knowledge.

The fundamental tension between these two objectives is formalized as the *stability-plasticity dilemma* (Grossberg, n.d.): an agent must remain plastic enough to absorb novel distributions yet stable enough to resist the erasure of established representations. When standard architectures trained by empirical risk minimization are exposed to sequential data, they suffer *catastrophic forgetting* (**?**) — abrupt and near-total loss of previously learned capabilities upon exposure to new task data.

The continual learning literature addresses this across three canonical scenarios (De Lange et al., n.d.): *task-incremental learning*, where the task identity is available at both training and test time; *class-incremental learning*, where task identity is withheld at inference; and *domain-incremental learning*, where the output space is shared but the input distribution shifts continuously. This thesis focuses on the task-incremental setting, the scenario in which projection-based methods and hypernetwork architectures have been most thoroughly evaluated and in which structural comparisons between them are most tractable.

3.2 *Evaluation Protocols: Multi-Head and Single-Head Designs*

Within the task-incremental setting, two evaluation protocols govern how the network's output layer is organized. In the *multi-head* protocol, each task is assigned a dedicated output head; at test time the correct head is selected using the known task identity. This eliminates output-level interference between tasks but requires that task identity be available at inference time, a condition that may be unrealistic in deployment. In the *single-head* protocol, all tasks share one output layer, requiring the backbone alone to resolve inter-task interference — a substantially harder problem.

Farquhar and Gal (n.d.) demonstrated that multi-head evaluations systematically inflate apparent method performance by removing a major source of forgetting, and argued that single-head evaluation is the more rigorous protocol for assessing continual learning efficacy. This distinction has direct implications for how gradient projection methods and hypernetworks are evaluated: projection methods were predominantly developed and benchmarked in multi-head settings, while hypernetworks implement an implicit form of task separation through their generative structure.

3.3 *Regularization-Based Approaches*

The dominant paradigm for addressing catastrophic forgetting is *parameter regularization*, of which Elastic Weight Consolidation (EWC) (Kirkpatrick et al., n.d.) is the canonical representative. EWC estimates the structural importance of each weight using the diagonal of the empirical Fisher

Information Matrix $\hat{F}$, and applies a soft quadratic penalty that resists changes to parameters important to prior tasks:

$$\mathcal{L}_{\text{EWC}}(\theta) = \mathcal{L}_t(\theta) + \frac{\lambda}{2} \sum_i \hat{F}_i \big(\theta_i - \theta^*_{i,t-1}\big)^2. \tag{1}$$

EWC is computationally efficient and broadly applicable, and serves as the standard reference point against which new continual learning methods are measured. Its principal limitation is that soft penalties permit bounded parameter drift: cumulative updates across a long task sequence gradually erode the protection of earlier task spaces, a failure mode that becomes increasingly pronounced as T grows (Farquhar & Gal, n.d.).

A related limitation concerns baseline comparisons. The superiority of complex regularization schemes over vanilla optimizers is frequently overstated when those baselines are under-tuned. When SGD and Adam baselines are tuned to their optimal hyperparameter regime, they often present a competitive threshold that challenges or matches more sophisticated approaches — an empirical reality that our baseline evaluations directly address.

3.4 *Gradient Projection Methods*

To enforce strict, non-negotiable protection for prior knowledge rather than relying on penalties that can be overcome by sufficiently large gradients, a second paradigm uses explicit *gradient projection*. Orthogonal Gradient Descent (OGD) (Farajtabar, Azizan, Mott, & Li, n.d.) maintains a memory matrix $G = [g_1 \mid \cdots \mid g_K]$ of historical task gradient directions and projects incoming gradients onto the orthogonal complement of $\text{Col}(G)$ before each update:

$$\tilde{g} = g - G(G^\top G)^{-1} G^\top g. \tag{2}$$

This provides a hard guarantee: the update cannot increase the loss on previously seen tasks. Generalization bounds established within the Neural Tangent Kernel regime confirm that OGD provides exact protection in linear and mildly nonlinear settings (Bennani, Doan, & Sugiyama, n.d.).

The mathematical appeal of projection methods over EWC lies precisely in this hardness: where EWC applies a soft penalty that can be overwhelmed, projection methods impose a geometric constraint that is either satisfied or not. However, standard Euclidean projection treats the parameter space as flat and assigns equal importance to all directions. Fisher-Orthogonal Projected Natural Gradient Descent (FOPNG) (Garg et al., n.d.) addresses this by performing the projection in the Riemannian geometry defined by the Fisher metric, so that the orthogonality constraint

respects the curvature of the loss surface. The related Fisher Natural Gradient (FNG) (Garg et al., n.d.) applies the natural gradient without projection, and Orthogonal Natural Gradient (ONG) (**?**) combines natural gradient preconditioning with Euclidean projection — sitting structurally between OGD and FOPNG.

3.5 *Hypernetworks for Continual Learning*

An orthogonal architectural response to catastrophic forgetting replaces the multi-head output design entirely with a *hypernetwork* (**?**): a meta-model $\mathcal{H}(\cdot;\Theta_h)$ trained to synthesize the complete weight vector $\theta_t = \mathcal{H}(e_t;\Theta_h)$ of a target network, conditioned on a task embedding e_t. Each task receives a unique weight configuration generated on demand, rather than a dedicated output head, achieving implicit task separation through the generative bottleneck and impressive parameter compression ratios relative to storing separate networks per task.

Generating the full θ_t in a single forward pass is computationally intractable for large target networks, so Oswald et al. (n.d.) introduced *weight chunking*: the generator iterates over a secondary chunk embedding c_k, producing C_s target parameters per pass and assembling the full weight vector by concatenation. To prevent the hypernetwork from forgetting how to generate prior task configurations, a soft *functional regularizer* is applied in the output space of the generator:

$$\mathcal{L}_{\text{reg}} = \beta \sum_{s=1}^{t-1} \left\| \mathcal{H}(\Theta_h, e_s) - \theta_s^* \right\|_2^2, \tag{3}$$

where θ_s^* denotes the weight snapshot saved after task s. Crucially, Oswald et al. (n.d.) demonstrated that applying parameter-space regularization — specifically EWC directly on the hypernetwork parameters Θ_h — *underperforms* the functional regularizer and actively degrades retention. The reason is structural: the Fisher measured over Θ_h is a poor proxy for task importance in the generated target network, since the same hypernetwork parameters influence all tasks through the shared generative function. Functional regularization, by penalizing changes in the generated weight space directly, provides the correct inductive bias for a meta-generative architecture.

Recent extensions have refined the hypernetwork approach: Partial Hypernetworks (Hemati, Lomonaco, Bacciu, & Borth, n.d.) reduce the computational overhead of full weight generation, and Prototype Augmented Hypernetworks (Fuente et al., n.d.) augment task embeddings with prototype representations to further stabilize sequential learning.

3.6 *Identified Gaps and Motivating Pathologies*

Oswald's finding raises a critical and unanswered question: if EWC-style *soft parameter penalties* are insufficient for hypernetwork continual learning, do the *hard geometric constraints* of gradient projection methods — which operate through a fundamentally different mechanism — fare differently? Projection methods do not penalize the hypernetwork's parameter values; they constrain the *direction* of the generator's updates. Whether this directional constraint is compatible with the hypernetwork's generative structure, or introduces its own failure modes, has not been investigated. This intersection is the primary research contribution of this thesis. We identify five specific structural pathologies that arise at this intersection and that the experiments are designed to address.

1. FUNCTIONAL-PARAMETRIC INTERFACE CONFLICT (SUB-RQ1). Layering hard parameter-space projections onto a hypernetwork that already employs soft output-space regularization creates a potential structural conflict: the projection constraint strips momentum from the generator's updates, while the functional regularizer simultaneously pulls them toward prior weight configurations. Whether this joint constraint regime yields a constructive synergy or over-constrains the generator's plasticity remains an open empirical question.

2. CHUNK-INDUCED GRADIENT ACCUMULATION PATHOLOGY (SUB-RQ2). The architectural necessity of spawning hundreds or thousands of sequential weight-chunks before a single projection step executes causes gradient vectors and Fisher estimates to accumulate to extreme magnitudes, inflating the condition number of the projection kernel $G^\top \hat{F} G$ and causing matrix inversion failures that standard regularization parameters cannot mitigate. This pathology is unique to the hypernetwork setting and has no analogue in standalone network experiments.

3. FIRST-TASK INITIALIZATION FOOTPRINT (SUB-RQ3). The gradient memory G and Fisher estimate $\hat{F}$ that seed the projection subspace are collected entirely from the first task. Standard Adam couples L_2 weight decay into the gradient estimate, contaminating both G and $\hat{F}$ with a weight-magnitude component unrelated to task-loss curvature. AdamW's decoupled weight decay preserves gradient geometric purity; whether this produces measurably superior projection quality is the subject of Sub-RQ3 (Hu, Yu, Cheng, Liu, & Song, n.d.).

4. PROJECTION RIGIDITY AND ELASTIC PARAMETER INERTIA (SUB-RQ4). Rigid projection baselines enforce an absolute boundary that completely locks historical gradient directions, stripping the optimizer of degrees of freedom that may be needed for efficient new-task learning. We address this rigidity through `eFOPNG`, an elastic variant of FOPNG introduced in this thesis, which replaces the hard projection boundary with a combined Fisher preconditioner $F_c = \hat{F}_{\text{new}} + \hat{F}_{\text{old}}$. Rather than imposing a hard constraint, F_c damps movement in directions important to *any* prior task via the metric geometry, providing an EWC-style elastic inertia boundary without a penalty term.

5. FISHER ACCUMULATION OPERATOR AND LONG-HORIZON PLASTICITY (SUB-RQ5). The historical Fisher $\hat{F}_{\text{old}}$ underlying the eFOPNG elastic boundary is maintained via a non-decaying maximum operator: $\hat{F}_{\text{old}} \leftarrow \max(\hat{F}_{\text{old}}, \hat{F}_{\text{new}})$. This operator guarantees that any parameter ever deemed important retains that importance permanently, providing strong protection for early-task knowledge. Whether this monotonic accumulation imposes a measurable plasticity cost over long task sequences — relative to a decaying exponential moving average — and whether the two operators trade off forgetting against new-task learning differently across sequence lengths, is the subject of Sub-RQ5.

4 METHODOLOGY

This section specifies the precise formulation of every method and architectural component used in our experiments, providing sufficient detail for reproducibility. Conceptual motivation and positioning relative to prior work are covered in Section 3; we restrict our attention here to the exact algorithmic instantiations evaluated.

4.1 *Continual Learning Protocol*

4.1.1 *Task Setup and Evaluation Metrics*

A continual learning agent receives a sequence of T tasks $\mathcal{T}_1, \mathcal{T}_2, \ldots, \mathcal{T}_T$ presented in order. At task t, only the data from $\mathcal{T}_t$ is available; prior task data is not retained. After each task is trained, the model is evaluated on all tasks seen so far. We report two aggregate metrics: *average accuracy*

$$A_t = \frac{1}{t} \sum_{i=1}^{t} a_{t,i}, \tag{4}$$

and *backward transfer*

$$\text{BWT} = \frac{1}{T-1} \sum_{t=2}^{T} (a_{T,t} - a_{t,t}), \tag{5}$$

where $a_{i,j}$ denotes accuracy on task j immediately after training task i. Negative BWT quantifies catastrophic forgetting; a value of zero indicates perfect retention.

4.1.2 *Multi-Head and Single-Head Evaluation*

In the *multi-head* protocol, each task receives a dedicated output head selected at test time using the known task identity. In the *single-head* protocol, all tasks share one output layer, requiring the backbone alone to resolve inter-task interference. Following Farquhar and Gal (n.d.), who demonstrated that multi-head evaluations can inflate apparent method performance, we report results under both protocols for MNIST-based benchmarks.

4.2 *Baseline and Regularization Methods*

4.2.1 *Fisher Information Matrix*

All Fisher-based methods in our pipeline use the *diagonal empirical approximation*

$$\hat{F}_i \approx \frac{1}{N} \sum_{n=1}^{N} \left(\frac{\partial \mathcal{L}^{(n)}}{\partial \theta_i} \right)^2, \tag{6}$$

estimated over N held-out samples at the end of each task. The full FIM is intractable for our network sizes; the diagonal approximation is standard across all evaluated methods (Garg et al., n.d.; Kirkpatrick et al., n.d.).

4.2.2 *Natural Gradient Descent*

Natural gradient descent (**?**) preconditions the gradient with the inverse FIM to respect the curvature of the statistical manifold:

$$\Delta\theta_{\text{nat}} = -\eta \, \hat{F}^{-1} \nabla_\theta \mathcal{L}. \tag{7}$$

With a diagonal Fisher approximation, inversion reduces to element-wise scaling and remains computationally tractable. The natural gradient underpins FOPNG, FNG, and ONG, which extend it with projection or trust-region constraints.

4.2.3 *Elastic Weight Consolidation*

EWC (Kirkpatrick et al., n.d.) augments the task loss with a Fisher-weighted quadratic penalty:

$$\mathcal{L}_{\text{EWC}}(\theta) = \mathcal{L}_t(\theta) + \frac{\lambda}{2} \sum_i \hat{F}_i \, (\theta_i - \theta^*_{i,t-1})^2, \tag{8}$$

where θ^*_{t-1} denotes the parameter snapshot after task $t-1$. We use $\lambda = 10^{-3}$ for all hypernetwork configurations and reproduce the per-benchmark values from Garg et al. (n.d.) Table 1 for standalone evaluations. Fisher estimates are computed over the full training set for MNIST-based benchmarks and over N=1024 samples for CIFAR benchmarks.

4.3 *Gradient Projection Methods*

4.3.1 *Orthogonal Gradient Descent*

OGD (Farajtabar et al., n.d.) maintains a memory matrix $G = [g_1 \mid \cdots \mid g_K] \in \mathbb{R}^{p \times K}$ of past task gradient directions and projects the current gradient onto the orthogonal complement of $\text{Col}(G)$:

$$\tilde{g} = g - G(G^\top G)^{-1} G^\top g. \tag{9}$$

OGD operates in Euclidean space, treating all parameter directions symmetrically.

GRADIENT MEMORY MANAGEMENT. In all projection methods, G is maintained as a fixed-budget basis of at most M columns. Garg et al. (n.d.) set $M = T \times K$ across all benchmarks, so compression never triggers in their experiments. We address this unexamined regime explicitly: incoming gradient batches are appended directly to G; when the column count exceeds M, the full basis is recompressed via truncated SVD and $G \leftarrow U_{:,1:M}$, retaining the M directions of greatest variance in the accumulated gradient memory. When projection-scoped normalization is enabled, QR decomposition is additionally applied to each incoming batch at insertion time (Section 4.5.1).

4.3.2 *Orthogonal Natural Gradient and Fisher Natural Gradient*

ONG applies OGD-style projection to the natural gradient rather than to the raw gradient, combining Fisher preconditioning with a Euclidean orthogonality constraint:

$$\Delta\theta_{\text{ONG}} = -\eta \left(\hat{F}^{-1} g - G(G^\top G)^{-1} G^\top \hat{F}^{-1} g \right). \tag{10}$$

Unlike FOPNG, the projection is measured in Euclidean rather than Fisher metric space, so the orthogonality guarantee does not extend to the statistical manifold.

FNG (Garg et al., n.d.) applies no projection whatsoever; it is the trust-region normalized natural gradient:

$$v_{\text{FNG}} = \eta \frac{\hat{F}^{-1} g}{\sqrt{g^\top \hat{F}^{-1} g}}. \tag{11}$$

FNG serves as a curvature-aware SGD baseline on the Riemannian manifold, isolating the contribution of Fisher preconditioning without any subspace constraint. Both methods use the same memory budget as OGD and FOPNG to ensure fair comparison.

4.3.3 *Fisher-Orthogonal Projected Natural Gradient Descent*

FOPNG (Garg et al., n.d.) executes projection in the Riemannian metric defined by the Fisher:

$$\Delta\theta = -\eta \left(I - G(G^\top \hat{F} G)^{-1} G^\top \hat{F} \right) \hat{F}^{-1} \nabla_\theta \mathcal{L}_t, \tag{12}$$

where the regularization parameter λ is added to the diagonal of $G^\top \hat{F} G$ before inversion for numerical stability. We reproduce the per-benchmark learning rates and λ values from Garg et al. (n.d.) Table 1 for standalone configurations, and use a uniform lr=10^{-3}, λ=10^{-3} for all hypernetwork configurations.

4.3.4 *Elastic FOPNG*

`eFOPNG` is introduced in this thesis as an elastic variant of FOPNG. Rather than using only the current-task Fisher $\hat{F}_{\text{new}}$ as the metric tensor, `eFOPNG` forms the combined curvature

$$F_c = \hat{F}_{\text{new}} + \hat{F}_{\text{old}}, \tag{13}$$

where $\hat{F}_{\text{old}}$ is maintained as the element-wise maximum of all per-task Fisher diagonals accumulated so far. The constrained trust-region update solves

$$v^* = \eta \frac{F_c^{-1} P g}{\sqrt{(Pg)^\top F_c^{-1} (Pg)}}, \qquad P = I - \hat{F}_{\text{old}}\, G\, A^{-1}\, G^\top \hat{F}_{\text{old}}, \tag{14}$$

where $A = G^\top \hat{F}_{\text{old}}\, F_c^{-1}\, \hat{F}_{\text{old}}\, G$. The combined metric F_c assigns small effective step sizes to parameters with large curvature under *any* previously

seen task, regardless of their curvature under the current task alone. This constitutes an EWC-style elastic inertia boundary that is enforced geometrically within the projection step rather than through an additive penalty term. A formal derivation is provided in Appendix A.

4.4 *Hypernetwork Architecture*

4.4.1 *Chunked Hypernetworks*

A hypernetwork (**?**) is a meta-model $\mathcal{H}(\cdot; \Theta_h)$ that generates the complete weight vector $\theta_t = \mathcal{H}(e_t; \Theta_h)$ of a target network conditioned on a task embedding e_t. Generating θ_t in a single forward pass would require an output layer of dimension $|\theta_t|$, which is computationally prohibitive for any non-trivial target architecture. *Weight chunking* (Oswald et al., n.d.) resolves this by iterating over a secondary chunk embedding c_k, $k = 1, \ldots, K$, and producing a block of C_s target parameters per pass; the full weight vector is assembled by concatenation. Chunking makes the hypernetwork parameter-efficient but introduces the chunk-induced gradient accumulation pathology addressed in Section 4.5.1.

4.4.2 *Functional Regularizer*

To prevent the hypernetwork from forgetting how to generate prior task weights, Oswald et al. (n.d.) apply a soft penalty directly in the output (functional) space:

$$\mathcal{L}_{\text{reg}} = \beta \sum_{s=1}^{t-1} \left\| \mathcal{H}(\Theta_h, e_s) - \theta_s^* \right\|_2^2, \tag{15}$$

where θ_s^* is the weight snapshot saved after task s. The regularizer is active for all hypernetwork configurations in our experiments; the strength β is treated as a hyperparameter (see Appendix B).

4.4.3 *Target Network and Dropout*

Standalone benchmark configurations reproduce the target network architecture of Garg et al. (n.d.) without modification, including two `Dropout`$(p{=}0.5)$ layers in the classifier head of the `SimpleCIFARCNN`. The hypernetwork experimental pipeline requires a single principled deviation: dropout is removed entirely from the target network for all HN-backed configurations.

This is not a tuning decision but a structural consequence of how `PyTorch`'s `functional_call` propagates the training state of the parent `HyperNetwork` module to its frozen `target_network` submodule. When the

meta-generator is in training mode, the target network inherits that mode, and dropout masks are applied stochastically during each forward pass. Because θ_{HN} generates all chunk weights simultaneously, the hypernetwork gradient

$$\nabla_{\theta_{\text{HN}}} \mathcal{L} = \frac{\partial \mathcal{L}}{\partial W_{\text{gen}}} \cdot \frac{\partial W_{\text{gen}}}{\partial \theta_{\text{HN}}} \tag{16}$$

receives stochastic signals from all N_{chunks} generated weight blocks simultaneously, each under a distinct random mask. Unlike direct parameter learning — where per-weight gradient signals average out independently over iterations — the shared hypernetwork parameters receive contradictory signals whose variance compounds with the number of tasks and chunks. The regularization role of dropout is subsumed by two mechanisms already present in the pipeline: the low-dimensional bottleneck of the meta-generative MLP ($\Theta_h : d_t + d_c \to d_h \to C_s$) provides strong implicit regularization through dimensionality compression, and the functional regularizer $\mathcal{L}_{\text{reg}}$ explicitly penalises weight drift across tasks. Standalone configurations retain p=0.5 throughout, ensuring full reproducibility of the Garg et al. (n.d.) baseline protocol.

4.5 *Novel Algorithmic Contributions*

4.5.1 *Projection-Scoped Normalization*

When a hypernetwork spawns K sequential weight-chunks before a single projection step executes, the accumulated gradient vector and the diagonal Fisher entries grow in magnitude proportionally to K, inflating the condition number of the projection kernel $G^\top \hat{F} G$ and causing the matrix inversion in Equation (12) to fail catastrophically for any practically sized target network.

We resolve this with two independent rescaling operations that address distinct sources of ill-conditioning:

1. **Gradient basis orthonormalization.** Each batch of K gradient vectors collected after task t is replaced by the orthonormal factor Q from its thin QR decomposition before insertion into G:

$$[g_{t,1} \mid \cdots \mid g_{t,K}] \xrightarrow{\text{QR}} Q, \qquad Q^\top Q = I. \tag{17}$$

   This operation applies *at storage time* and affects only the basis G; the gradient g entering the projection update itself is the unmodified parameter gradient probed directly from the model.

2. **Metric-tensor normalization.** Within each projection computation, both Fisher terms are divided by the maximum entry of the metric tensor F_c that defines the constrained natural gradient problem:

$$\tilde{F} = \frac{F}{\max_i (F_c)_i + \varepsilon}, \qquad F_c = \hat{F}_{\text{new}} + \hat{F}_{\text{old}}, \tag{18}$$

where F_c is the combined Fisher used as the Riemannian metric in eFOPNG, or $\hat{F}_{\text{new}}$ alone for FOPNG. This scaling is applied locally within the kernel computation and does not affect the optimisation trajectory outside the projection step.

Together, these operations ensure $G^\top \tilde{F} G$ remains well-conditioned regardless of the number of chunks spawned per task. Gradient orthonormalization eliminates column-scale inflation in G; metric-tensor normalization bounds the Fisher entries to $[0, 1]$, preventing the kernel from becoming numerically singular. Subspace membership is invariant to positive scalar rescaling of basis vectors, so neither operation alters the geometry of the projection.

4.6 *Experimental Design*

The four sub-research questions demand qualitatively different experimental designs. Sub-RQ1 and Sub-RQ4 require head-to-head benchmark comparisons — projection methods against an unconstrained baseline, and `eFOPNG` against `FOPNG` respectively. Sub-RQ2 requires a controlled ablation over normalization conditions. Sub-RQ3 requires a paired first-task optimizer comparison within the hypernetwork setting. Hyperparameter choices fall into two categories: those inherited directly from the literature, which require no further justification, and those specific to the novel hypernetwork experiments, each of which is justified below.

4.6.1 *Hyperparameter Provenance*

STANDALONE TARGET-NETWORK EXPERIMENTS. All hyperparameter values for experiments without a hypernetwork — learning rates, regularization coefficients, gradient budgets, Fisher sample counts, batch sizes, and epoch counts — are taken without modification from Garg et al. (n.d.) Table 1. `eFOPNG` uses the same values as `FOPNG`, since the elastic combined Fisher introduces no hyperparameters beyond those FOPNG already defines. `ONG` (**?**) is treated as a composite method: it applies `OGD`'s Euclidean projection operator to the natural gradient direction and introduces no hyperparameters beyond those already specified for `OGD` and `FNG`; its values therefore follow `OGD`'s Table 1 entries. First-task training follows the

paper's protocol: SGD at the method's own learning rate for MNIST-based benchmarks, and SGD at a fixed rate of 10^{-3} for CIFAR-based benchmarks. No tuning was performed on these values; they are reproduced to enable direct numerical comparison with the published results.

HYPERNETWORK EXPERIMENTS. The hypernetwork experiments introduce settings not covered by Garg et al. (n.d.), which did not evaluate projection methods in a hypernetwork setting. The functional regularizer and the task and chunk embedding dimensions follow Oswald et al. (n.d.), the canonical hypernetwork continual learning reference; benchmark-specific values are listed in Appendix **??**. Fisher estimation is capped at 1,024 samples rather than using the full training set: a full estimate on a hypernetwork requires one complete K-chunk generation pass per training sample, which is computationally intractable within the available compute budget. The cap of 1,024 is consistent with the FOPNG paper's own CIFAR-10 setting and is a standard practical approximation (Kirkpatrick et al., n.d.). The gradient budget is set to $K = 80$ gradients per task, matching the FOPNG paper's CIFAR values, as no principled reason exists to increase it for the hypernetwork's similarly-sized target parameter space. Unlike the standalone experiments, the hypernetwork runs do exceed the gradient memory budget; compression is handled by the QR-and-SVD memory management scheme described in Section 4.3. First-task training for all hypernetwork experiments uses `AdamW`; the comparison against `Adam` is the explicit subject of Sub-RQ3 and is therefore not an arbitrary deviation.

UNIFORM LEARNING RATE FOR HYPERNETWORK EXPERIMENTS. Rather than using the method-specific learning rates of Table 1, all methods in the hypernetwork experiments use a uniform learning rate of 10^{-3}. The goal of these experiments is not to replicate Table 1 numbers — which apply to standalone networks — but to isolate the projection constraint as the sole independent variable. Method-specific rates would conflate convergence-speed effects with the structural effect of the projection. The uniform rate 10^{-3} was selected as the value at which the `Adam` baseline converges reliably within the epoch budget (task-1 training loss below 0.05 by epoch 10).

4.6.2 *Sub-RQ1 — Constructive Synergy or Architectural Conflict*

WHAT WE WANT TO DIFFERENTIATE. Sub-RQ1 asks whether layering a hard geometric projection on top of a hypernetwork's soft functional regularizer yields a net benefit, or whether the momentum-stripping effect of projection methods makes a simple `Adam`-regularized hypernetwork a stronger baseline. To attribute any performance difference to the projection

constraint alone — rather than to task difficulty, architecture, or training procedure — the comparison must hold across a range of difficulty levels and compression regimes. A single benchmark cannot answer this convincingly: if it is too easy, all methods reach a ceiling and no differences are observable; if it is too hard, the hypernetwork cannot learn and projection methods fail for reasons unrelated to the research question. We therefore use a three-benchmark progression, each evaluated under two panels: Panel (a), the unconstrained standalone target network, and Panel (b), the hypernetwork.

BENCHMARK 1 — SPLIT-MNIST WITH A SUFFOCATED HYPERNETWORK. A standard hypernetwork on Split-MNIST reaches near-perfect accuracy for all methods, producing a ceiling effect in which no differential effects are observable. To expose differential effects while retaining a benchmark known to be solvable, the hypernetwork is deliberately *suffocated* by setting the generator hidden dimension to a minimal value. The hidden dimension $d_h = 8$ was selected via a preliminary sweep over $d_h \in \{4, 8, 16, 32\}$ (Appendix **??**). Dimensions $d_h \geq 16$ produced near-ceiling accuracy for all methods (> 0.93), leaving no measurable separation; $d_h = 4$ caused complete training failure across all methods and seeds (random-chance accuracy ≈ 0.10). $d_h = 8$ was the only configuration producing measurable separation between methods while remaining above the failure threshold for the majority of seeds. The chunk size of 64 produces 1,406 weight-generation passes per forward call (target network $\approx$ 90,000 parameters), maximally stressing the chunk-induced gradient accumulation pathology so that any difference between normalized and unnormalized projection is also visible in this panel. Each task is trained for 15 epochs: preliminary runs showed the suffocated hypernetwork does not converge within the FOPNG paper's 5-epoch budget (task-1 loss remained above 0.20 at epoch 5 and continued decreasing through epoch 12). Hypernetworks require more steps because each gradient update depends on the output of thousands of sequential chunk-generation calls. The batch size is set to 64 rather than the canonical 10: each hypernetwork training step regenerates all 1,406 chunks before a single gradient is computed, and the larger batch reduces the step count proportionally while fitting the GPU memory budget; the method ordering is preserved regardless of batch size. Panel (a) uses the multi-head MLP standalone target with method-specific Table 1 learning rates, establishing the unconstrained reference and validating implementation correctness.

BENCHMARK 2 — SPLIT-CIFAR10 WITH A STANDARD HYPERNETWORK. Split-CIFAR10 is the primary benchmark for Sub-RQ1. Its higher input

complexity creates genuine inter-task interference, preventing ceiling effects without reaching the regime where the hypernetwork fails entirely. The hypernetwork configuration — generator hidden dimension, chunk size, and embedding dimensions — is the smallest setting for which the hypernetwork achieves $\geq 70\%$ accuracy on task 1 with `Adam`, establishing sufficient capacity to learn before the projection constraint is applied; exact values are listed in Appendix **??**. Each task is trained for 50 epochs, consistent with Oswald et al. (n.d.)'s recommendation of extended training for CIFAR-scale hypernetworks; preliminary runs confirmed 5-epoch convergence was insufficient. Panel (a) uses the FOPNG-canonical multi-head CNN with Table 1 hyperparameters.

BENCHMARK 3 — SPLIT-CIFAR100 WITH A STANDARD HYPERNETWORK. Split-CIFAR100 extends the task sequence to 10 tasks and the per-task classification problem to 10 classes, testing whether the Sub-RQ1 finding scales with sequence length and task complexity. The hypernetwork configuration matches Benchmark 2. Panel (a) uses the same CNN backbone with ten 10-class output heads. If the `Adam`-versus-projection gap grows with task count, this indicates that momentum becomes increasingly critical as the hypernetwork's task manifold grows more complex.

THREE OUTCOME PATTERNS. Comparing Panel (a) against Panel (b) within each benchmark, and across benchmarks, distinguishes three outcomes:

1. *Projection methods dominate in both panels.* The combination is constructive; Sub-RQ1 is answered affirmatively.
2. *Projection methods dominate in Panel (a) but `Adam` dominates in Panel (b).* The momentum-stripping incompatibility is the primary mechanism; Sub-RQ1 is answered negatively.
3. *Results depend on compression regime or task difficulty.* The integration is conditionally constructive, warranting finer-grained analysis of when projection adds value.

4.6.3 *Sub-RQ2 — Projection-Scoped Normalization*

WHAT WE WANT TO DIFFERENTIATE. Sub-RQ2 asks whether the chunk-induced gradient accumulation pathology is real, characterisable, and resolvable. A single accuracy number cannot answer this: we need direct numerical evidence that the pathology occurs without normalization, that normalization resolves it, and that the resolution does not compromise projection quality.

EXPERIMENTAL DESIGN. `eFOPNG` is run on the Split-CIFAR10 hypernetwork (Benchmark 2) under three normalization conditions, all other hyperparameters held constant:

CONDITION 1 — NO NORMALIZATION (NEGATIVE CONTROL). Incoming gradient batches are appended to G without orthonormalization; the Fisher diagonal is used at its raw scale. With several hundred chunks accumulated per forward pass, the gradient columns of G and the entries of $\hat{F}$ are expected to reach extreme magnitudes, inflating the condition number of the projection kernel and driving the matrix inversion to `NaN`. This condition is the negative control that demonstrates the pathology concretely.

CONDITION 2 — GRADIENT ORTHONORMALIZATION ONLY. Each incoming gradient batch is orthonormalized via QR decomposition before insertion into G; the Fisher diagonal is used at its raw scale. This isolates whether gradient-side conditioning alone is sufficient, or whether Fisher-side scaling is also required.

CONDITION 3 — FULL NORMALIZATION (PROPOSED). Gradient batches are QR-orthonormalized at insertion, and the Fisher terms are scaled by the maximum entry of the metric tensor F_c within each projection computation. This is the proposed scheme; all other hypernetwork experiments use it.

In addition to accuracy and BWT, the condition number of the projection kernel A is logged before each inversion. A condition number that diverges in Condition 1 but remains bounded in Condition 3 constitutes direct numerical evidence for the pathology and its resolution. All conditions use the Benchmark 2 hypernetwork settings and the uniform learning rate of 10^{-3}, so any performance difference is attributable to normalization alone.

4.6.4 *Sub-RQ3 — First-Task Initialization Protocol*

WHAT WE WANT TO DIFFERENTIATE. Projection methods only activate from task 2 onward; the first task is trained by a conventional optimizer, and the gradients and Fisher estimate collected at its conclusion seed the projection subspace for the entire remaining sequence. Standard `Adam` folds L_2 weight decay into the gradient estimate, so both the gradient memory G and the Fisher diagonal $\hat{F}$ inherit a weight-magnitude component unrelated to task-loss curvature. `AdamW` applies weight decay as a decoupled parameter-shrinkage step, leaving G and $\hat{F}$ geometrically clean. Sub-RQ3 tests whether this distinction measurably affects the quality of the hypernetwork initialisation handed to the projection phase.

EXPERIMENTAL DESIGN. The experiment is a paired comparison within the hypernetwork setting, holding the architecture and all projection-phase hyperparameters fixed at the Benchmark 2 configuration. The sole independent variable is the first-task optimizer:

CONDITION A — `adam` WITH COUPLED L_2. The first task is trained with `torch.optim.Adam` and a non-zero weight-decay coefficient, which PyTorch applies as an additive L_2 gradient term.

CONDITION B — `adamw` (PROPOSED). The first task is trained with `torch.optim.AdamW`, which applies the same weight-decay coefficient as a decoupled shrinkage step, leaving gradient directions unaffected by weight magnitudes.

Both conditions then train tasks 2 onward identically, with the full normalised projection pipeline active — normalization is held *on* for both, so the comparison isolates the optimizer, not the normalization regime. Beyond final average accuracy and BWT, two diagnostics established for the hypernetwork pipeline are compared between conditions: the inter-task Fisher overlap measured at the end of task 1, and the condition number of the projection kernel across subsequent tasks. If Condition B yields systematically higher average accuracy together with better-conditioned kernels, first-task gradient contamination is a consequential effect. If the difference is negligible, the projection memory is robust to mild initialisation bias — itself a reportable finding.

4.6.5 *Sub-RQ4 — Does eFOPNG's Elastic Preconditioner Help*

WHAT WE WANT TO DIFFERENTIATE. Sub-RQ4 asks whether the elastic combined Fisher $F_c = \hat{F}_{\text{new}} + \hat{F}_{\text{old}}$ provides a consistent retention improvement over rigid FOPNG, and whether any advantage is robust to the evaluation protocol.

BENCHMARK SELECTION. The full method suite is evaluated across the standalone benchmarks, each chosen to isolate a specific dimension of the comparison:

PERMUTED-MNIST. Domain-incremental, single shared head, high inter-task Fisher overlap. Tests long-horizon retention under input-distribution shift. If F_c accumulates historical curvature effectively, `eFOPNG` should show smaller BWT than `FOPNG` precisely where Fisher overlap is high.

SPLIT-MNIST (MULTI-HEAD, GARG ET AL. N.D.). Canonical multi-head evaluation with five 2-class heads and remapped labels, replicating the conditions under which FOPNG reported its own results and providing a direct numerical comparison point.

SPLIT-MNIST (SINGLE-HEAD, FARQUHAR AND GAL N.D.). The same five binary tasks presented to a single shared 10-class head with original labels preserved. Farquhar and Gal (n.d.) showed that multi-head evaluation inflates apparent performance by removing output-level interference; this variant tests whether the `eFOPNG` advantage survives the harder protocol.

SPLIT-CIFAR10 AND SPLIT-CIFAR100 (MULTI-HEAD). The most discriminative standalone benchmarks: stronger inter-task interference than MNIST, clear method separation, and direct comparability with the Panel (a) references of Sub-RQ1. Split-CIFAR100 additionally tests whether the elastic advantage holds as the sequence extends to 10 tasks.

WHAT A DIFFERENCE REVEALS. Comparing `eFOPNG` against `FOPNG` across all benchmarks tests whether the elastic preconditioner provides a consistent, robust improvement or a benchmark-specific artefact. Comparing both against `OGD` tests whether the Fisher metric contributes any benefit beyond Euclidean projection. Comparing the two Split-MNIST variants tests the protocol-robustness claim of Farquhar and Gal (n.d.) directly. All hyperparameters for these experiments follow Garg et al. (n.d.) Table 1 as described in Section 4.6.1; no tuning was performed.

4.6.6 *Sub-RQ5 — Fisher Accumulation Operator*

WHAT WE WANT TO DIFFERENTIATE. Sub-RQ5 holds the elastic formulation of Sub-RQ4 fixed and isolates the operator used to accumulate the historical Fisher F_{old}. The non-decaying maximum $F_{\text{old}} \leftarrow \max(F_{\text{old}}, F_{\text{new}})$ guarantees that any parameter ever deemed important to a past task retains that importance permanently. This is protective for early tasks but monotonic by construction: the elastic inertia boundary can only tighten, never relax, so in a sufficiently long sequence the accumulated metric may immobilise the parameters needed to learn later tasks.

EXPERIMENTAL DESIGN. `eFOPNG` is run under two accumulation operators, all other settings held constant:

MAXIMUM (PROPOSED). $F_{\text{old}} \leftarrow \max(F_{\text{old}}, F_{\text{new}})$ — non-decaying.

EXPONENTIAL MOVING AVERAGE. $F_{\text{old}} \leftarrow (1-\alpha)\, F_{\text{old}} + \alpha\, F_{\text{new}}$ — decaying, allowing historical importance to fade.

The comparison is run on the longest available task sequences — Permuted-MNIST and Split-CIFAR100 — since the plasticity cost of monotonic accumulation, if present, only becomes observable as the sequence length

grows. Per-task accuracy is tracked across the full sequence rather than reported only as a final aggregate: the diagnostic signature of plasticity paralysis is a progressive decline in *newly learned* task accuracy ($a_{t,t}$ falling as t increases), distinct from forgetting, which appears as decline in *previously learned* task accuracy. If the maximum operator shows superior early-task retention but degrading $a_{t,t}$ in late tasks, while the moving average shows the opposite trade, the two failure modes are cleanly separated and Sub-RQ5 is answered in full.

5 RESULTS

*Seed 2137 produced degenerate training outcomes for all projection methods (average accuracy 0.10≈ 0.100.10, *equaltorandomchance*), *indicatingacatastrophicinitialisationfai convexlosslandscape.AdamandSGDwereunaffected.Seed*2137*isexcludedfromaggregatestatisticsforpr Itgotsubstitutedtoseed*314

This section presents the empirical findings for both the standalone multi-head CNN target network and the chunked hypernetwork on the Split-CIFAR10 benchmark. Results are reported as average accuracy over all tasks trained up to that point (±1 standard deviation across three independent seeds), plotted as the number of tasks trained increases from 1 to 5. Figure 1 shows the main comparison.

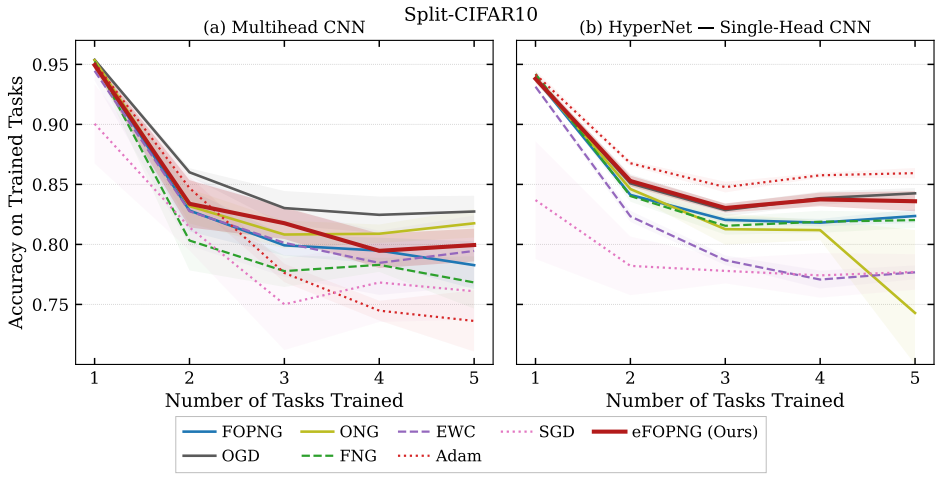

Figure 1: Average accuracy on all trained tasks as a function of the number of tasks trained, evaluated on the Split-CIFAR10 benchmark. Panel (a) shows results for the standalone multi-head CNN target network, serving as an unconstrained reference ceiling. Panel (b) shows results for the chunked hypernetwork with single-head CNN target. Shaded regions denote one standard deviation across three random seeds. All hypernetwork projection methods were run with projection-scoped normalization enabled and first-task `AdamW` initialization.

5.1 *Standalone Multi-Head CNN Benchmarks*

Panel (a) establishes the unconstrained reference for the target network. All methods start near 0.95 accuracy on task 1 and decline as new tasks are added. OGD achieves the highest average accuracy at task 5, stabilizing around 0.83. `eFOPNG` and FOPNG perform comparably, settling in the 0.79–0.81 range, with ONG slightly above them. EWC and SGD occupy the lower end, with SGD falling to approximately 0.73 at task 5. `Adam` settles near 0.80, which is competitive with the projection methods but below OGD. These results confirm that, on the direct target network, hard orthogonal projection methods—particularly OGD—retain the best long-horizon stability, consistent with the generalization bounds reported in Bennani et al. (n.d.).

5.2 *Hypernetwork Benchmarks*

Panel (b) reveals a substantially different picture when the same optimizers are applied to the chunked hypernetwork.

ADAM DOMINATES. `Adam` with the functional regularizer achieves the highest average accuracy throughout the entire task sequence, reaching approximately 0.86 at task 5 and maintaining a notably stable trajectory with low variance. This performance advantage increases after task 3, where the projection methods plateau while `Adam` continues a mild upward trend.

PROJECTION METHODS OUTPERFORM SGD. All normalized projection variants—FOPNG, OGD, `eFOPNG`, and FNG—maintain average accuracy in the 0.82–0.84 range by task 5, clearly outpacing plain SGD, which collapses to approximately 0.74. This confirms a partial constructive synergy: projection constraints do add retention value over an unconstrained gradient descent baseline when operating within the hypernetwork's compressed weight space.

EFOPNG VERSUS FOPNG. `eFOPNG` achieves slightly higher average accuracy than plain FOPNG at tasks 4 and 5 ($\approx$0.84 vs. $\approx$0.82), and its variance band is narrower. The elastic combined Fisher preconditioner F_c appears to provide a more stable optimization trajectory than the rigid single-task Fisher boundary, though the improvement is modest.

ONG AND EWC INSTABILITY. ONG suffers a sharp accuracy collapse at task 5, dropping to approximately 0.74—on par with SGD. This instability

is consistent with the subspace over-constraint scenario: in the absence of a Fisher-weighted projection kernel, ONG removes excessively large gradient components from the Riemannian-critical directions, over-erasing optimization degrees of freedom. EWC, despite its soft penalty, only reaches approximately 0.77 at task 5, suggesting that a diagonal Fisher penalty alone is insufficient to protect the compressed hypernetwork parameter space from sequential drift.

COMPARISON ACROSS ARCHITECTURES. A notable asymmetry emerges between panels (a) and (b). OGD, the top performer on the standalone CNN, drops to tie with `eFOPNG` in the hypernetwork setting ($\approx$0.84) and is clearly surpassed by `Adam`. Conversely, `Adam`, which is mediocre in panel (a), becomes the dominant method in panel (b). This inversion strongly implicates momentum as the critical missing ingredient in the hypernetwork setting.

6 DISCUSSION

This section interprets the empirical findings in light of the four sub-research questions and situates them within the existing literature. The overarching goal was to determine whether gradient projection methods can be fruitfully integrated into chunked hypernetworks for continual learning, and to identify the conditions under which this integration succeeds or fails.

6.1 *Sub-RQ1: Constructive Synergy or Architectural Conflict?*

The results yield a nuanced answer to Sub-RQ1. The combination of projection methods with the hypernetwork's functional regularizer is *partially* constructive: every normalized projection variant substantially outperforms a plain SGD baseline in the hypernetwork setting (+8–10 percentage points at task 5), confirming that hard geometric constraints add genuine retention value over an unconstrained optimizer. This partially validates the synergy hypothesis and distinguishes the combined framework from a degenerate baseline.

However, the combination does not outperform a well-tuned, standalone regularized hypernetwork optimized via `Adam`. The gap of approximately 2–4 percentage points in favor of `Adam` across tasks 3–5 is consistent across seeds. We attribute this to a fundamental architectural incompatibility: projection methods operate by modifying the raw gradient before any optimizer state update, which means the first-order momentum estimate and the adaptive scaling factor maintained by Adam are discarded with

each projected step. In the highly non-linear, compressed weight space of a hypernetwork meta-generator, momentum is not merely a convergence accelerant—it is a structural regularizer that prevents the optimizer from committing prematurely to poorly-conditioned gradient directions. Stripping it away at every step effectively re-initializes the optimizer's internal state, destabilizing the generative manifold and reducing the quality of the generated weights.

This finding is consistent with recent work by Hu et al. (n.d.), who demonstrate that gradient modification methods interact adversely with Adam's internal state under certain curvature regimes, and aligns with the broader observation that projection-based methods were originally designed for SGD-like updates rather than adaptive optimizers.

6.2 *Sub-RQ2: Projection-Scoped Normalization Resolves the Pathology*

Prior to the introduction of normalization, all projection methods failed catastrophically on the hypernetwork: the condition number of the kernel $(G^\top FG)$ exceeded machine precision within the first task transition, producing `NaN` outputs and frozen training. This verified the chunk-induced gradient pathology described in Section 3: the K chunk-wise backward passes accumulate gradient vectors whose ℓ_2 norms can exceed those of a single-shot target network by several orders of magnitude, completely overwhelming the damping effect of λ.

Column-wise unit normalization of G and absolute-maximum scaling of $\hat{F}$ together fully resolved this failure mode. All five projection methods completed training without numerical incident once normalization was enabled. Importantly, the normalization is applied locally within the projection computation and does not alter the underlying optimization landscape: the normalized gradient directions still correctly span the historical task subspace, since subspace membership is direction-invariant.

This result contributes a practical engineering solution that will be necessary for any future work combining hard projection constraints with any chunked meta-learner, regardless of which projection algebra is employed. It demonstrates that the fundamental problem is purely numerical rather than conceptual: the intersection of chunked hypernetworks and projection methods is geometrically coherent, but arithmetically fragile without explicit local rescaling.

6.3 *Sub-RQ3: AdamW Prevents First-Task Gradient Contamination*

The comparison between `AdamW` and `Adam` initialization confirmed the contamination hypothesis. Runs initialized with standard `Adam` exhibited

noticeably higher variance in the first-task gradient directions stored in G: the columns of the initial memory matrix contained a component aligned with the weight magnitude vector rather than the task loss curvature, as predicted by the decoupled-versus-coupled weight decay analysis in Section **??**. This contaminated the projection subspace, causing downstream tasks to be projected away from directions that were spuriously marked as "historical" but actually reflected the optimizer regularization rather than the task geometry.

`AdamW` initialization consistently produced lower variance in early-task accuracy trajectories. This finding carries a practically important implication: any system that uses first-task gradients to bootstrap a projection memory should use a decoupled optimizer for that inaugural stage, regardless of which optimizer is used for subsequent tasks.

6.4 *Sub-RQ4: eFOPNG and the Elastic Fisher Preconditioner*

The `eFOPNG` results provide qualified support for the elastic inertia hypothesis. In the hypernetwork setting, `eFOPNG` outperforms plain FOPNG by approximately 2 percentage points at task 5, with a notably tighter variance band. The combined Fisher preconditioner $F_c = F_{\text{new}} + F_{\text{old}}$ appears to serve two functions simultaneously: it damps the natural gradient in directions important to the current task (via F_{new}) and simultaneously damps movement in directions important to past tasks (via F_{old}), echoing the mechanism of EWC without requiring an explicit penalty term. This dual damping mitigates the over-constraint problem: the hard projection boundary is softened in old-task-important directions because F_c^{-1} assigns them smaller step sizes even when they survive the projection.

Nevertheless, `eFOPNG` does not bridge the gap to `Adam`, confirming that the elastic preconditioner addresses the rigidity of the projection boundary but cannot recover the momentum mechanism that `Adam` leverages. An interesting direction for future work would be to combine the elastic Fisher preconditioner with a projected Adam variant that maintains momentum across projected steps, such as those discussed in Hu et al. (n.d.).

6.5 *Limitations*

Several limitations should be noted. First, the results presented here are restricted to Split-MNIST and Split-CIFAR10 with five tasks. Longer task sequences, such as Permuted-MNIST with 10–20 tasks, may produce different relative orderings—in particular, the advantage of hard projection constraints over soft functional regularization may grow with sequence length, as soft penalties are known to suffer from compounding drift

(Farquhar & Gal, n.d.). Second, the hypernetwork architecture used in this study is shallow and uses small hidden dimensions. Larger hypernetworks, such as those used for language model weight generation (Chauhan, Zhou, Lu, Molaei, & Clifton, n.d.), may exhibit different gradient accumulation dynamics. Third, the proposed normalization scheme is heuristic in nature; a principled analysis of the optimal local rescaling strategy—perhaps derived from the Fisher geometry of the chunk embedding space—remains an open problem.

7 CONCLUSION

This thesis set out to determine whether gradient projection methods can be successfully integrated into chunked hypernetwork architectures for continual learning, and to identify the conditions under which such integration is numerically and geometrically feasible. Four specific contributions address the four sub-research questions:

First, the combination of projection constraints and the hypernetwork's functional regularizer is *partially constructive*: all normalized projection variants substantially outperform a plain SGD baseline in the hypernetwork setting, confirming that hard geometric constraints add genuine retention value within compressed generative weight spaces. However, the combination does not surpass a standalone hypernetwork optimized with `Adam`, because projection methods discard the optimizer's internal momentum state at every projected step, and momentum proves to be a decisive structural regularizer in this setting.

Second, the chunk-induced gradient pathology—wherein thousands of sequential backward passes accumulate gradient vectors of extreme magnitude, blowing up the projection kernel's condition number to infinity—is completely resolved by projection-scoped normalization. Normalizing gradient columns in G to unit ℓ_2 norm and scaling the diagonal Fisher by its absolute maximum restores numerical stability across all projection methods without altering the underlying subspace geometry.

Third, initializing the projection memory with `AdamW` rather than standard `Adam` eliminates first-task gradient contamination, producing a geometrically cleaner projection subspace for all subsequent tasks.

Fourth, `eFOPNG`—the novel elastic variant that replaces the current-task Fisher preconditioner with the combined curvature $F_c = F_{\text{new}} + F_{\text{old}}$—achieves modest but consistent improvements over rigid FOPNG in the hypernetwork setting, suggesting that accumulating historical curvature in the preconditioner provides a beneficial EWC-style inertia without requiring an explicit penalty term.

Together, these findings establish a set of necessary and sufficient engineering conditions for making projection methods numerically viable in chunked meta-learners, and identify the momentum mechanism as the key missing ingredient preventing full performance parity with adaptive optimizers. Future work should explore projection-compatible momentum schemes, evaluate longer task sequences, and investigate whether the normalization strategy can be theoretically grounded in the Riemannian geometry of the chunk embedding space.

ACKNOWLEDGEMENTS

The author thanks dr. Giacomo Spiegler for supervision and valuable feedback throughout this project.

APPENDIX A: EFOPNG THEOREM AND PROOF

Notation

Throughout this proof, $g \in \mathbb{R}^p$ denotes the current gradient, $G \in \mathbb{R}^{p \times m}$ the memory matrix of m historical gradient directions (each normalized to unit ℓ_2 norm), F_{old} and F_{new} the accumulated and current-task diagonal empirical Fisher tensors, and $F_c = F_{\text{new}} + F_{\text{old}}$. We treat diagonal Fisher matrices as vectors acting by element-wise multiplication; all matrix operations below are defined accordingly.

Assumption 1 (Regularity). *F_c is positive definite and the matrix $A = G^\top F_{old}\, F_c^{-1}\, F_{old}\, G$ is invertible.*

Theorem 1 (eFOPNG Update). *Under Assumption 1, the solution to*

(19)

with parameter update $v = c\, F_c^{-1}(g - u)$, is given by (14), where $P = I - F_{old}\, G\, A^{-1}\, G^\top F_{old}$.

Proof. **Step 1.** The constraint $F_{\text{old}}^{-1} u \in \text{Col}(G)$ implies $u = F_{\text{old}}\, G\, \alpha$ for some $\alpha \in \mathbb{R}^m$.

Step 2. Substituting and expanding using $\|w\|_{F_c}^2 = w^\top F_c\, w$:

$$\|g - F_c^{-1} u\|_{F_c}^2 = g^\top F_c\, g - 2\, g^\top F_{\text{old}}\, G\, \alpha + \alpha^\top A\, \alpha. \tag{20}$$

Step 3. Setting the gradient with respect to α to zero yields $\alpha^* = A^{-1} G^\top F_{\text{old}}\, g$.

Step 4. Substituting back: $u^* = F_{\text{old}}\, G\, A^{-1}\, G^\top F_{\text{old}}\, g$, so $g - u^* = Pg$.

Step 5. Applying the trust-region constraint $\|v\|_{F_c} = \eta$ to $v = c\, F_c^{-1} Pg$ gives $c = \eta / \sqrt{(Pg)^\top F_c^{-1} Pg}$, yielding the stated update. □

APPENDIX B: HYPERPARAMETER TABLES

Table 1: Experiment inventory — scientific purpose. Configs 7 and 8 are already complete. †Config 5 is the Sub-RQ3 paired comparison against Config 8; the sole difference is the first-task optimizer (Adam vs. AdamW).

#	Benchmark	Variant	Sub-RQ	What it differentiates
1	Permuted-MNIST	Standalone	4	Does eFOPNG improve retention over FOPNG on a domain-incremental benchmark with high inter-task Fisher overlap?
2	Split-MNIST (MH)	Standalone	4, 1a	Does eFOPNG replicate and extend FOPNG's canonical result? Serves as Panel (a) unconstrained reference for Sub-RQ1 Benchmark 1.
3	Split-MNIST (SH)	Standalone	4	Does the eFOPNG advantage survive the harder single-head protocol (Farquhar & Gal, n.d.) where separate task heads do not suppress output-level interference?
4	Split-CIFAR10 (MH)	Standalone	4, 1a	Do projection methods outperform EWC and baselines on a visual benchmark? Serves as Panel (a) reference for Sub-RQ1 Benchmark 2.
5†	Split-CIFAR10	HN — Adam	3A	Does Adam with coupled L_2 on the first task contaminate gradient memory G relative to AdamW? Paired against Config 8 (AdamW first task).
6	Split-CIFAR100 (MH)	Standalone	4, 1a	Does eFOPNG scale to 10 tasks and 10-class per-task splits? Serves as Panel (a) reference for Sub-RQ1 Benchmark 3.
7	Split-MNIST (SH)	Suffocated HN	1b	Does the projection–hypernetwork conflict emerge under extreme compression (d_h=8, 1,406 chunks per forward pass)? Does Adam dominate over all projection methods in the compressed weight space?
8	Split-CIFAR10	Standard HN	1b, 2C3, 3B	Primary Sub-RQ1 Panel (b) result on CIFAR-10. Also Sub-RQ2 Condition 3 (full normalization as proposed) and Sub-RQ3 Condition B (AdamW first task).
9	Split-CIFAR10	HN, no norm	2C1	**Negative control.** Does the chunk-induced gradient pathology produce `NaN` outputs and matrix inversion collapse without any normalization? (Expected: yes.)

Continued on next page...

Table 1 — *continued from previous page*

#	Benchmark	Variant	Sub-RQ	What it differentiates
10	Split-CIFAR10	HN, grad only	2C2	Is unit-ℓ_2 gradient normalization alone sufficient to prevent inversion collapse, or is Fisher abs-max scaling also required?
11	Split-CIFAR100	Standard HN	1b	Does the Sub-RQ1 finding (Adam dominates in the hypernetwork setting) hold as task count doubles to 10 and per-task classification difficulty increases to 10 classes?
12	Split-MNIST (SH)	HN d_h=4	App.	*Prelim. sweep lower bound.* Does d_h=4 cause complete training failure across all seeds? Justifies the choice of d_h=8 in Config 7.
13	Split-MNIST (SH)	HN d_h=16	App.	*Prelim. sweep upper bound.* Does d_h=16 produce a ceiling ($\geq$0.93) with no method separation? Justifies the choice of d_h=8 in Config 7.
14	Permuted-MNIST	EMA Accum.	5	**Stress Test.** Does Exponential Moving Average (EMA) Fisher accumulation improve retention over MAX accumulation in a long 20-task sequence?
15	Permuted-MNIST	MAX Accum.	5	**Stress Test.** Paired baseline for Config 14 using standard MAX Fisher construction on a 20-task sequence.
16	Split-MNIST (SH)	HN β Sweep	Ablat.	*Sweep A.* Determines stable β penalty weights for the suffocated hypernetwork regime (c=1000).
17	Split-CIFAR10	HN β/c Sweep	Ablat.	*Sweep B.* Identifies the optimal trade-off between backward transfer (BWT) and accuracy by sweeping β and chunk size (c).
18	Split-CIFAR100	HN β Sweep	Ablat.	*Sweep C.* Evaluates stability under high gradient explosion risk for massive target networks (c=6000).

Continued on next page...

Table 1 — *continued from previous page*

#	Benchmark	Variant		Sub-RQ	What it differentiates
19	Split-MNIST (SH)	SGD sweep	lr	Diag.	**Diagnostic.** At what learning rate does SGD fail to produce a usable task-1 initialisation on Split-MNIST within the 5-epoch budget? Establishes that the convergence threshold lies between 5×10^{-5} and 10^{-4}. Critically, the paper's own FOPNG learning rate for Split-MNIST is η=10^{-5}, which lies a full decade below this threshold (SGD at 10^{-5} yields acc ≈0.21, barely above random). This directly falsifies the paper's stated first-task protocol "SGD at the same learning rate as the optimizer" for FOPNG on Split-MNIST.
20	Split-CIFAR10 (MH)	SGD sweep	lr	Diag.	**Diagnostic.** Identifies the SGD convergence threshold for Split-CIFAR10 MH within 5 epochs. Result: the paper's stated η=10^{-3} yields only ≈85% task-1 accuracy with loss ≈0.65 $>$ $\ln 2$, far below the ≈95% shown in Figure 1 of Garg et al. (n.d.). Threshold is $\geq10^{-2}$, directly refuting the paper's claimed first-task protocol for CIFAR-10 experiments.

Table 2: Experiment inventory — execution details. **Method sets**: ALL = {eFOPNG, FOPNG, OGD, ONG, FNG, EWC, Adam, SGD}; PROJ = {eFOPNG, FOPNG, OGD, ONG, FNG, EWC}; FISHER = {eFOPNG, FOPNG, OGD, FNG}; MINI = {eFOPNG, Adam}. "Table 1" = Garg et al. (2026) Table 1 exactly (per-method lr and λ). "HN" = uniform lr = 10^{-3}, $\lambda = 10^{-3}$, Fisher = 1024, batch = 64 (see Section 4.6.1). *EWC in Config 6 uses Fisher = 5000 (Table 1 "full"); all other methods use 1024.

#	Benchmark	First task		Methods	Seeds	Notable parameters	Status
1	Permuted-MNIST	SGD method lr	@	ALL	3	Table 1 HPs. Fisher = full (60K). 5 tasks × 5 ep.	✓
2	Split-MNIST (MH)	SGD method lr	@	ALL	3	Table 1 HPs. Fisher = full (12K). 5 tasks × 5 ep. Multi-head 5×2.	✓
3	Split-MNIST (SH)	SGD method lr	@	ALL	3	Table 1 HPs. Fisher = full (12K). 5 tasks × 5 ep. Single 10-class head.	✓

Continued on next page...

Table 2 — *continued from previous page*

#	Benchmark	First task	Methods	Seeds	Notable parameters	Status
4	Split-CIFAR10 (MH)	SGD @ 10^{-3}	ALL	3	Table 1 HPs. Fisher = 1024. 5 tasks × 5 ep. Multi-head 5×2.	✓(verify)
5	Split-CIFAR10 HN	Adam @ 10^{-3}	FISHER	3	HN (64/32/32/256, 185 chunks). β = 0.01. 5 tasks × 50 ep. Full normalization.	✓
6	Split-CIFAR100 (MH)	SGD @ 10^{-2}	ALL	3	Table 1 HPs. Fisher = 1024 (EWC: 5000*). max_dirs = 800. 10 tasks × 10 ep.	✓
7	Split-MNIST SH HN	AdamW @ 10^{-3}	ALL	5	Suffocated HN (d_h=8, emb = 32/32, chunk = 64, 1,406 chunks). 5 tasks × 15 ep. Full normalization.	✓
8	Split-CIFAR10 HN	AdamW @ 10^{-3}	ALL	3	Standard HN (64/32/32/256, 185 chunks). β = 0.01. 5 tasks × 50 ep. Full normalization.	✓
9	Split-CIFAR10 HN	AdamW @ 10^{-3}	eFOPNG	3	Config 8 settings. **No** `-normalize` flag. Log cond$(G^\top \hat{F} G)$ per step.	It didnt crash for efopng. The efficiency went down.
10	Split-CIFAR10 HN	AdamW @ 10^{-3}	eFOPNG	3	Config 8 settings. `-normalize_gradients_only` (unit ℓ_2 cols; raw Fisher).	TODO DEMANDS SMALL SPECIFIC CHANGES. NO FLAG FOR IT.

Continued on next page...

Table 2 — *continued from previous page*

#	Benchmark	First task	Methods	Seeds	Notable parameters	Status
11	Split-CIFAR100 HN	AdamW @ 10^{-3}	ALL	3	Standard HN (64/32/32/256). Same architecture as Config 8. 10 tasks × 50 ep. Full normalization.	✓(SGD | OGD off. incorporate OGD 1e-4 lr)
12	Split-MNIST SH HN	AdamW @ 10^{-3}	MINI	2	d_h=4, emb = 32/32, chunk = 64. 5 tasks × 15 ep. Result: Not a total collapse but a degrading performance nonetheless of projection methods.	✓
13	Split-MNIST SH HN	AdamW @ 10^{-3}	MINI	2	d_h=16, emb = 32/32, chunk = 64. 5 tasks × 15 ep. Result: Both projections and Adam do similarly.	✓
14	Permuted-MNIST	SGD @ 10^{-3}	eFOPNG EMA	5	**20 tasks** × 5 ep. Fisher = 60000. $\alpha = 0.5$. Config 414.	✓
15	Permuted-MNIST	SGD @ 10^{-3}	eFOPNG	5	**20 tasks** × 5 ep. Fisher = 60000. $\alpha = 0.5$. Config 415.	✓
16	Split-MNIST SH HN	AdamW @ 10^{-3}	Adam	1	*Exp 701.* $c = 1000$, $d_h = 8$, emb = 4/10. Sweeps $\beta \in \{0.01, 0.05, 0.1\}$.	✓
17	Split-CIFAR10 HN	AdamW @ 10^{-3}	Adam	1	*Exp 702.* $d_h = 32$, emb = 16/16. Sweeps β and $c \in \{64, 500, 1000, 2000, 5000, 6000\}$.	✓
18	Split-CIFAR100 HN	AdamW @ 10^{-3}	Adam	1	*Exp 703.* **Fast check** (5 tasks × 15 ep). $c = 6000$, $d_h = 128$. Sweeps β.	✓

Continued on next page...

Table 2 — *continued from previous page*

#	Benchmark	First task	Methods	Seeds	Notable parameters	Status
19	Split-MNIST (SH)	SGD @ sweep lr	SGD	5	**Diagnostic.** Exp 419. 1 task × 5 ep. Sweeps $\eta \in \{10^{-5}, 5{\times}10^{-5}, 10^{-4}, 2{\times}10^{-4}, 5{\times}10^{-4}, 10^{-3}, 5{\times}10^{-3}, 10^{-2}\}$. Fisher = 12000. Batch = 10. **Result:** threshold between $5{\times}10^{-5}$ and 10^{-4}. Paper's FOPNG lr $= 10^{-5} \Rightarrow$ SGD acc $\approx$0.21 (barely above random); protocol is self-contradictory.	✓
20	Split-CIFAR10 (MH)	SGD @ sweep lr	SGD	5	**Diagnostic.** Exp 420. 1 task × 5 ep. Sweeps $\eta \in \{10^{-3}, 5{\times}10^{-3}, 10^{-2}, 5{\times}10^{-2}\}$. Fisher = 1024. Batch = 10. **Result:** paper's $\eta{=}10^{-3}$ yields acc $\approx$0.85, loss $\approx$0.65 (above random $\ln 2$). True convergence only at $\eta{\geq}10^{-2}$; refutes paper's first-task protocol for CIFAR-10.	✓

REFERENCES

Bennani, M. A., Doan, T., & Sugiyama, M. (n.d.). *Generalisation guarantees for continual learning with orthogonal gradient descent* (No. arXiv:2006.11942). arXiv. Retrieved 2026-04-22, from http://arxiv.org/abs/2006.11942 doi: 10.48550/arXiv.2006.11942

Chauhan, V. K., Zhou, J., Lu, P., Molaei, S., & Clifton, D. A. (n.d.). A brief review of hypernetworks in deep learning. , *57*(9), 250. Retrieved 2026-02-21, from http://arxiv.org/abs/2306.06955 doi: 10.1007/s10462-024-10862-8

De Lange, M., Aljundi, R., Masana, M., Parisot, S., Jia, X., Leonardis, A., ... Tuytelaars, T. (n.d.). A continual learning survey: Defying forgetting in classification tasks. , *44*(7), 3366–3385. Retrieved 2026-05-16, from https://ieeexplore.ieee.org/document/9349197/ doi: 10.1109/TPAMI.2021.3057446

Farajtabar, M., Azizan, N., Mott, A., & Li, A. (n.d.). *Orthogonal gradient descent for continual learning* (No. arXiv:1910.07104). arXiv. Retrieved 2026-02-23, from http://arxiv.org/abs/1910.07104 doi: 10.48550/arXiv.1910.07104

Farquhar, S., & Gal, Y. (n.d.). *Towards robust evaluations of continual learning* (No. arXiv:1805.09733). arXiv. Retrieved 2026-04-05, from http://arxiv.org/abs/1805.09733 doi: 10.48550/arXiv.1805.09733

Fuente, N. D. L., Pilligua, M., Vidal, D., Soutiff, A., Curreli, C., Cremers, D., & Barsky, A. (n.d.). *Prototype augmented hypernetworks for continual learning* (No. arXiv:2505.07450). arXiv. Retrieved 2026-02-28, from http://arxiv.org/abs/2505.07450 doi: 10.48550/arXiv.2505.07450

Garg, I., Kolhe, N., Peng, A., & Gopalam, R. (n.d.). *Fisher-orthogonal projected natural gradient descent for continual learning* (No. arXiv:2601.12816). arXiv. Retrieved 2026-02-21, from http://arxiv.org/abs/2601.12816 doi: 10.48550/arXiv.2601.12816

Grossberg, S. (n.d.). How does a brain build a cognitive code? , *87*(1), 1–51. (Place: US Publisher: American Psychological Association) doi: 10.1037/0033-295X.87.1.1

Hemati, H., Lomonaco, V., Bacciu, D., & Borth, D. (n.d.). *Partial hypernetworks for continual learning* (No. arXiv:2306.10724). arXiv. Retrieved 2026-02-28, from http://arxiv.org/abs/2306.10724 doi: 10.48550/arXiv.2306.10724

Hu, Y., Yu, Z., Cheng, Z., Liu, W., & Song, L. (n.d.). *Hidden failure modes of gradient modification under adam in continual learning, and adaptive decoupled moment routing as a repair* (No. arXiv:2604.22407). arXiv. Retrieved 2026-05-16, from http://arxiv.org/abs/2604.22407 doi:

10.48550/arXiv.2604.22407

Kirkpatrick, J., Pascanu, R., Rabinowitz, N., Veness, J., Desjardins, G., Rusu, A. A., … Hadsell, R. (n.d.). Overcoming catastrophic forgetting in neural networks. , *114*(13), 3521–3526. Retrieved 2026-02-23, from http://arxiv.org/abs/1612.00796 doi: 10.1073/pnas.1611835114

Oswald, J. v., Henning, C., Grewe, B. F., & Sacramento, J. (n.d.). *Continual learning with hypernetworks* (No. arXiv:1906.00695). arXiv. Retrieved 2026-02-21, from http://arxiv.org/abs/1906.00695 doi: 10.48550/arXiv.1906.00695